

Text Processing and NLP Encoding

Atoms: An Architectural Blueprint

The Paradigm of Pure Functions in Computational Directed Graphs

The evolution of Natural Language Processing (NLP) architectures has increasingly gravitated toward modular, deterministic pipelines, often formalized as Computational Directed Graphs (CDGs). Within these frameworks, the integrity of the entire system rests upon the reliability of its foundational processing primitives, or "atoms." Whether deploying classical machine learning algorithms via scion-atoms-ml or orchestrating large-scale transformer architectures via scion-atoms-dl, the demand for computational purity is absolute.

A pure function in this context is defined by stringent boundaries: it maps explicit input types—such as strings, primitive arrays, or lists—to explicit output artifacts without relying on global state, hidden random number generators, or external network calls. Furthermore, these atoms must avoid in-execution file I/O, ensuring that any necessary assets (such as frequency dictionaries or model weights) are loaded immutably into the module's scope prior to execution. By enforcing these constraints, pipelines achieve idempotency, allowing for aggressive caching, horizontal scaling, and deterministic debugging.

This exhaustive research report delineates the optimal algorithms, licensing implications, mathematical formulations, and technical specifications required to standardize these NLP atoms. Priority is given to highly optimized, lightweight implementations released under permissive MIT or BSD licenses, ensuring rapid execution and frictionless integration into enterprise-grade CDGs.

Text Cleaning and Normalization Mechanisms

Raw text data acquired from heterogeneous sources—such as web scraping, optical character recognition (OCR), or unconstrained human input—is inherently noisy. In predictive modeling contexts ranging from Kaggle's Jigsaw Toxicity to the Mercari Price Suggestion Challenge, text normalization acts as the critical first layer of defense.¹ The objective is to collapse high-cardinality syntactical noise into standardized lexical representations, reducing the out-of-vocabulary (OOV) burden on downstream tokenizers.

Deterministic Regular Expression Filtering

The initial stages of text processing leverage pre-compiled regular expressions to excise or isolate non-informative artifacts. Data gathered from the web universally requires the neutralization of HTML tags, URLs, and residual punctuation artifacts.⁴ The compilation of these regex patterns at the module level is critical for performance, as it amortizes the cost of finite

state automaton construction, ensuring $O(N)$ evaluation complexity over the character length of the document.⁴

Standard operations include the substitution of URL patterns `r'https?://\S+'` and HTML tags `r'<.*?>'` with empty strings, followed by the normalization of whitespace using `r'\s+'`.⁵

Furthermore, the isolation of punctuation is frequently required to prevent standard alphanumeric words from fusing with adjacent punctuation marks, which would otherwise generate unique, spurious tokens in unigram models.

The Imperative of Unicode Normalization

Beyond superficial character filtering, text normalization must address the deep complexities of Unicode encoding. Visually identical glyphs can be represented by vastly different underlying byte sequences. For instance, the character "é" can be encoded as a single precomposed character (U+00E9) or as a decomposed sequence of "e" (U+0065) followed by a combining acute accent (U+0301). Without standardization, these representations hash to divergent integer IDs, fracturing the embedding space.

The application of Normalization Form Compatibility Composition (NFKC) is universally recommended for NLP preprocessing.⁵ NFKC goes beyond canonical equivalence by decomposing characters into their fundamental semantic components—including compatibility characters like full-width alphanumeric letters or typographic ligatures (e.g., decomposing "fi" into "f" and "i")—and subsequently recomposing them into standard, uniform glyphs. This deterministic mapping drastically reduces the sparsity of the token vocabulary.

Probabilistic Spelling Correction via Algorithmic Edit Distance

While modern deep learning architectures often absorb minor typographical errors through subword tokenization, classical NLP pipelines and specific extraction tasks benefit immensely from explicit spelling correction. However, invoking a massive, machine-learning-based spell checker introduces unacceptable latency and non-deterministic behavior that violates CDG constraints. The optimal solution is an algorithmic, dictionary-based approach, famously pioneered by Peter Norvig.⁷

The algorithm approaches spelling correction as a probabilistic inference problem utilizing Bayes' Theorem. To find the most probable intended correction c for a given misspelled word w , the algorithm maximizes the probability:

$$\arg \max_c P(c|w) = \arg \max_c \frac{P(w|c) \cdot P(c)}{P(w)}$$

Because the probability of the misspelled word $P(w)$ is constant across all candidate corrections, it can be safely ignored, simplifying the objective to $\arg \max_c P(w|c) \cdot P(c)$.⁸

In this deterministic framework, $P(c)$ represents the prior probability of the corrected word appearing in the language, which is retrieved via an $O(1)$ lookup from a pre-computed unigram frequency dictionary.⁹ The term $P(w|c)$ represents the error model, which quantifies the likelihood of the author typing w when they intended c .

Rather than computing $P(w|c)$ across the entire dictionary—which would be computationally catastrophic—the algorithm generates candidates structurally. The input word expands sequentially into permutations via four fundamental operations: deletions (removing one letter), transpositions (swapping adjacent letters), replacements (changing one letter), and insertions (adding one letter). For a word of length L , this generates $54L + 25$ unique candidates at an edit distance of 1.⁸ These candidate permutations are immediately filtered against the valid frequency dictionary. The process is then repeated to generate candidates at an edit distance of 2. Finally, the surviving valid candidates are scored by their dictionary frequency, and the optimal correction is emitted.⁷

Python libraries such as `pyspellchecker` encapsulate this exact logic purely and efficiently under an MIT license, completely avoiding network calls or hidden model states.⁷ To preserve the pure-function boundary, the dictionary must be explicitly passed into the atom or instantiated mutably at module load time.

Name: `clean_text`

Description: Normalizes raw text strings by stripping HTML/URLs, applying Unicode NFKC normalization, isolating punctuation, and optionally correcting spelling via dictionary lookup.

Source: <https://norvig.com/spell-correct.html> (Norvig algorithm reference)

License: MIT

Concept type: `data_extraction`

Signature: `(text: str, operations: list[str], vocab_freq: dict[str, int] = None) -> str`

Pure function boundary: strings and strict operational parameters in, normalized string out; no network calls, regex caching occurs at module load, no external file I/O during execution.

Contracts:

- require: `operations` is a subset of `{"html", "url", "unicode", "lower", "spell"}`
- require: if `"spell"` in `operations`, `vocab_freq` must be provided
- ensure: output contains no non-printable control characters

Witness: Input `"Visitt http://example.com/ Now!"`, Operations `["html", "url", "lower", "spell"]`, Output `"visit now!"`

Dependencies: pure Python (`re`, `unicodedata`)

CDG stages covered: jigsaw/text_cleaning, mercari/normalization

String Distance and Similarity Metrics

Feature engineering in tabular NLP pipelines frequently requires quantifying the morphological divergence between strings. This is paramount in tasks such as Foursquare Location Matching or entity resolution, where predicted strings must be aligned against ground-truth categorical features. The core algorithmic primitives powering these computations are the Levenshtein distance and the Jaro-Winkler similarity.

Levenshtein Distance Computations

The Levenshtein distance provides an absolute integer metric quantifying the minimum number of single-character edits required to mutate a source string into a target string.¹⁰ The permissible edit operations are insertions, deletions, and substitutions, each carrying a uniform cost of 1.

Mathematically, the distance between two strings a and b of lengths N and M is defined by the recursive piecewise function:

$$\begin{aligned} lev_{a,b}(i, j) &= \max(i, j) \text{ if } \min(i, j) = 0 \\ lev_{a,b}(i, j) &= \min \begin{cases} lev_{a,b}(i-1, j) + 1 \\ lev_{a,b}(i, j-1) + 1 \\ lev_{a,b}(i-1, j-1) + 1_{(a_i \neq b_j)} \end{cases} \quad \text{otherwise} \end{aligned}$$

A naive recursive implementation of this equation yields exponential time complexity, rendering it useless for production CDGs. Standard implementations employ the Wagner-Fischer algorithm, a dynamic programming approach that constructs an $(N+1) \times (M+1)$ matrix, populating the minimal edit costs iteratively.¹² This results in $O(NM)$ time and space complexity. However, because the computation of any cell (i, j) depends only on the current row and the immediately preceding row, memory-optimized implementations discard older rows, reducing the space complexity to $O(\min(N, M))$.

While pure Python implementations utilizing this optimized two-row DP approach operate adequately on short strings—averaging roughly 14.4 microseconds per computation¹¹—the high-throughput demands of processing millions of records necessitate C-compiled extensions. Historically, the python-Levenshtein and fuzzywuzzy libraries dominated this space; however, their restrictive GPL licensing renders them toxic for many proprietary or enterprise CDG deployments.¹³

The state-of-the-art solution is the rapidfuzz library.¹³ Licensed under MIT, rapidfuzz utilizes highly optimized C++ bindings that drastically outperform legacy libraries.¹⁶ Furthermore, its memory allocation strategy is engineered to achieve near-zero cache misses (L1, LLi, LLd, LL) by utilizing contiguous memory blocks and minimizing heap allocations.¹⁶ For the purpose of establishing a pure-function NLP atom, rapidfuzz provides the definitive, drop-in backend.

Name: levenshtein

Description: Calculates the absolute minimum edit distance between two strings using optimized dynamic programming.

Source: <https://github.com/rapidfuzz/RapidFuzz>

License: MIT

Concept type: analysis

Signature: (s1: str, s2: str) -> int

Pure function boundary: parallel strings in, single integer distance out.

Contracts:

- require: s1 and s2 are valid string types
- ensure: result >= 0
- ensure: result <= max(len(s1), len(s2))
- ensure: levenshtein(s1, s2) == levenshtein(s2, s1)

Witness: "kitten", "sitting" -> 3

Dependencies: rapidfuzz (MIT C-extension) or pure Python fallback DP implementation.

CDG stages covered: foursquare/string_similarity

Jaro-Winkler Similarity and Prefix Scaling

While the Levenshtein algorithm provides an absolute integer distance, the Jaro-Winkler algorithm provides a normalized similarity score bounded continuously between **0.0** and **1.0**, where **1.0** dictates exact equivalency.¹⁷ This metric was explicitly designed for record linkage and entity resolution, excelling at matching proper nouns and short strings.

The foundational Jaro distance J computes the weighted sum of matching characters and transpositions.¹⁷ Crucially, two characters from string s_1 and s_2 are only considered a "match" if they are identical and their positional index difference does not exceed a strict search

window: $\lfloor \frac{\max(|s_1|, |s_2|)}{2} \rfloor - 1$.¹⁷

The Jaro equation is defined as: $J = \frac{1}{3} \left(\frac{m}{|s_1|} + \frac{m}{|s_2|} + \frac{m-t}{m} \right)$ where m is the number of

matching characters within the bounded window, and t is half the number of matching characters that are out of sequence (transpositions).¹⁷

Winkler's seminal modification to this algorithm applies a prefix bonus.¹⁹ This adjustment operates on the empirical observation that humans rarely make typographical errors at the very beginning of a word, and therefore strings sharing a common prefix have a significantly higher probability of true equivalence.²⁰

The final Jaro-Winkler score J_w scales the Jaro score upward based on prefix alignment:
 $J_w = J + \ell p(1 - J)$ where ℓ is the length of the common prefix (capped at a maximum of 4 characters) and p is a constant scaling factor (standardized at **0.1**).¹⁸

As with the Levenshtein metric, the rapidfuzz library provides the most computationally aggressive, MIT-licensed implementation of Jaro-Winkler.¹⁴ A fallback pure Python implementation requires meticulous indexing to track the matching characters within the sliding window, bounded by $O(NM)$ time complexity in the worst case but practically operating much faster due to the windowing constraint.¹⁷

String Similarity Metric Comparison

Metric	Output Range	Complexity	Prefix Weighted	Optimal Use Case
Levenshtein	Integer count (Minimum number of edits)	$O(nm)$ one row DP. Builds a matrix grid step-by-step.	No	• Spell checking & comparing OCR text • Fuzzy matching in databases • Handling non-Latin characters
Jaro	Floating point score (Typically 0.0 to 1.0)	Calculation based on matching and transposed characters within a defined sliding window distance.	No	Comparing two short strings where spelling errors are more prone to happen at the end of the string.
Jaro-Winkler	Floating point score (Typically 0.0 to 1.0)	Jaro score calculation + Winkler's factor to increase score based on common prefix sequence.	Yes (Impact is limited to first 4 characters)	When early string mismatches should be penalized heavily. Ideal for data where typos generally occur later in the text string.

Comparison of primary string distance metrics. RapidFuzz implementations provide optimal C-level execution speeds while maintaining the MIT license.

Data sources: [Stack Overflow](#), [IJEEDU](#), [PyPI \(fuzzmatch\)](#), [Medium](#), [PyPI \(pyjarowinkler\)](#), [Mohit Mayank](#), [Interactive Brokers](#)

Name: jaro_winkler

Description: Calculates the prefix-weighted string similarity score tailored for entity and name resolution.

Source: <https://github.com/rapidfuzz/RapidFuzz>

License: MIT

Concept type: analysis

Signature: (s1: str, s2: str, prefix_weight: float = 0.1) -> float

Pure function boundary: strings in, float similarity score out.

Contracts:

- require: prefix_weight <= 0.25 (to analytically prevent score from exceeding 1.0)
- ensure: result >= 0.0
- ensure: result <= 1.0
- ensure: jaro_winkler(s, s) == 1.0

Witness: "martha", "marhta" -> 0.9611

Dependencies: rapidfuzz (MIT) or pure Python fallback.

CDG stages covered: foursquare/string_similarity

Sequence Labeling and Tag Decoding Topologies

Named Entity Recognition (NER), Semantic Role Labeling, and part-of-speech tagging mandate that machine learning models emit sequences of categorical tags mapping 1:1 with input tokens.²⁴ To synthesize these atomic token-level predictions back into contiguous, actionable entity spans (e.g., extracting discourse elements in Kaggle's Feedback Prize competition), the CDG requires a mathematically robust tag decoding atom.

Transition Logic in BIO and BILOU State Machines

The widely accepted industry standard for evaluating sequence labeling tasks is the seqeval framework.²⁴ Embedded within its core metrics module is the pure Python decoding logic required to translate arrays of tags into a list of consolidated span tuples formatted as (entity_type, start_index, end_index).

The decoding process operates as a deterministic finite state automaton transitioning across the tag array.²⁷ Models typically output tags conforming to specific topological schemes, predominantly BIO (Begin, Inside, Outside) or the more granular BILOU (Begin, Inside, Last, Outside, Unit) formats.²⁶

The state machine implements two distinct boundary detection functions at every index i :

1. **Boundary Termination (end_of_chunk):** An active contiguous entity span is instantly terminated under several combinatorial conditions.²⁷ The explicit condition triggers if the preceding tag was E (End) or S (Single/Unit). The implicit condition triggers if an active span encounters an Outside tag (O), a new Begin tag (B-), or a Unit tag (U-). Crucially, a type mismatch also forces termination—for example, transitioning directly from I-PER to I-LOC signifies the abrupt end of a Person entity and the immediate commencement of a Location entity.
2. **Boundary Initialization (start_of_chunk):** A new entity span is initialized explicitly when the iterator encounters a Begin (B-) or Unit (S-/U-) tag.²⁷

Strict vs. Default Evaluation Heuristics

A critical architectural consideration in tag decoding is the handling of ill-formed sequences. Deep learning models, especially during early training epochs or under out-of-domain inference conditions, may generate grammatically impossible tag sequences, such as emitting an I-ORG tag without a preceding B-ORG tag.

In its default mode, the sequeval heuristic engine is highly permissive, mimicking legacy Perl scripts like conllevl. If an I- tag appears autonomously, the state machine retroactively assumes a chunk initialization, gracefully recovering from the model error and attempting to salvage the span.²⁷ Conversely, in strict mode, the automaton aggressively penalizes structural violations, immediately invalidating the span and collapsing the chunk.²⁷ For pure-function extraction atoms in a production CDG, the permissive default mode is generally preferred to maximize recall, provided the downstream logic can filter out noisy, truncated spans.

Name: bio_decode

Description: Convert an aligned sequence of BIO/BIOU tags and textual tokens into labeled span tuples via deterministic finite automata.

Source:

https://github.com/chakki-works/sequeval/blob/master/sequeval/metrics/sequence_labeling.py

License: MIT

Concept type: data_assembly

Signature: (tags: list[str], tokens: list[str]) -> list[tuple[str, int, int]]

Pure function boundary: parallel lists of strings in, extracted tuples of (entity_type, start_idx, end_idx) out.

Contracts:

- require: len(tags) == len(tokens)
- require: tags adhere to standard IOB1, IOB2, IOE1, IOE2, IOBES, or BIOU string prefixes
- ensure: returned span indices satisfy $0 \leq \text{start} \leq \text{end} < \text{len}(\text{tokens})$
- ensure: returned tuples are strictly ordered by start_idx

Witness: Tags, Tokens ->

Dependencies: pure Python (adapted directly from sequeval core logic)

CDG stages covered: feedback_prize/bio_decode, kaggle_ner/span_extraction

Subword Tokenization and Offset Alignment

The proliferation of transformer architectures fundamentally shifted the NLP paradigm from word-level tokenization to subword algorithmic tokenization, specifically utilizing Byte-Pair Encoding (BPE), WordPiece, and SentencePiece.²⁸ These algorithms systematically merge

high-frequency character pairs into cohesive subword tokens, perfectly balancing vocabulary size against out-of-vocabulary (OOV) minimization.

The Thin Wrapper Paradigm

Implementing BPE or WordPiece tokenization entirely from scratch in pure Python is computationally untenable. The continuous string manipulation, dictionary lookups, and iteration loops trigger severe Python Global Interpreter Lock (GIL) bottlenecks, resulting in execution times hundreds of times slower than compiled equivalents.²⁹

The optimal architectural design for the tokenization atom is a "thin wrapper" paradigm. HuggingFace's tokenizers library is fundamentally constructed in Rust, executing heavily parallelized character mapping and byte-level alignment natively in memory. It entirely bypasses the GIL.²⁹ The NLP atom should wrap `AutoTokenizer.from_pretrained()`, explicitly stripping away all heavyweight PyTorch or TensorFlow framework dependencies. The pure function simply accepts a string and yields a pure Python dictionary of integer arrays (specifically `input_ids` and `attention_mask`).³⁰

Binary Search for Character-to-Token Offset Alignment

In extraction tasks such as the Kaggle Feedback Prize, ground-truth labels are provided as character-level spans (e.g., characters 15 through 42) directly on the raw document.²⁸ However, transformer models natively operate on, and output predictions for, token indices. Bridging this gap requires precise alignment.

HuggingFace's fast, Rust-backed tokenizers expose an `offset_mapping` tensor.²⁷ This array comprises tuples representing the (`char_start`, `char_end`) boundaries of every generated subword token relative to the original document string.

The `char_to_token_offsets` atom maps the original continuous character spans back to discrete token indices. Because the tokenization process is strictly sequential, the `offset_mapping` array is guaranteed to be monotonically increasing with respect to character positions.²⁷

Consequently, an $O(N)$ linear scan across the entire document to find matching offsets is deeply inefficient.

The optimal implementation leverages this monotonicity by utilizing a binary search algorithm (via Python's built-in `bisect` module). Given a ground truth character span [`C_start`, `C_end`], the atom performs a binary search over the `offset_mapping` array to locate the precise `token_start`

index that bounds C_{start} and the `token_end` index that bounds C_{end} . This reduces the

alignment complexity from linear to $O(\log T)$ per entity, where T is the number of tokens in the document, proving essential for processing massive corpora.

Name: `char_to_token_offsets`

Description: Aligns original document character spans to transformer subword token

boundaries utilizing binary search over offset mapping arrays.

Source: Derived from standard Kaggle Feedback Prize top solutions

License: MIT

Concept type: data_assembly

Signature: (char_spans: list[tuple[int,int]], offset_mapping: list[tuple[int,int]]) -> list[tuple[int,int]]

Pure function boundary: integer tuples in, mapped token index tuples out.

Contracts:

- require: offset_mapping is strictly monotonically increasing
- require: char_spans elements (start, end) where start < end
- ensure: returned token indices fall entirely within range(len(offset_mapping))
- ensure: len(output) == len(char_spans)

Witness: char_spans [(0, 4)], offset_mapping [(0, 2), (2, 4)] -> [(0, 1)]

Dependencies: pure Python (bisect) or numpy

CDG stages covered: feedback_prize/char_to_token

Sequence Decoding via Beam Search

In sequence-to-sequence generation tasks, such as machine translation or the BMS Molecular Translation dataset, the model acts autoregressively, predicting the next discrete token based on the joint probability of all preceding tokens.³⁶ Selecting only the single highest-probability token at each arbitrary timestep—known as greedy decoding—frequently traps the generation trajectory in local optima, producing suboptimal, truncated, or nonsensical sequences.³⁷

Conversely, exhaustive search across the entire vocabulary space is mathematically impossible, requiring $O(|V|^T)$ evaluations. Beam search decoding occupies the optimal middle ground, maintaining the top- k most probable hypotheses (where k is the beam width) at every timestep, expanding the search space linearly to $O(k \cdot |V|)$.³⁶

Framework-Agnostic State Invariants

To remain viable as a pure function within a CDG, the beam search implementation must be strictly decoupled from the underlying deep learning framework (e.g., PyTorch, TensorFlow, JAX). The logic must accept a generic log_probs_fn callable.³⁶ This external function accepts an array of current token sequences and returns the log-probabilities of the next vocabulary distribution, entirely encapsulating the model's complex forward pass, KV-cache updates, and hardware state.⁴⁰

Operating strictly in log-probability space is mandatory; multiplying raw probabilities over hundreds of autoregressive steps guarantees catastrophic floating-point underflow.³⁶ The

implementation loop orchestrates the search via explicit Numpy invariants³⁶:

1. ACTIVE_SEQ: An integer tensor storing the current top- k sequences that have not yet terminated.
2. ACTIVE_LOG_PROBS: A float tensor tracking the cumulative log-probabilities of those sequences.
3. FINISHED_SEQ: An array sequestering hypotheses that have successfully generated the End-of-Sequence (EOS_ID) token.
4. FINISHED_SCORES: The final probability scores of the completed sequences.

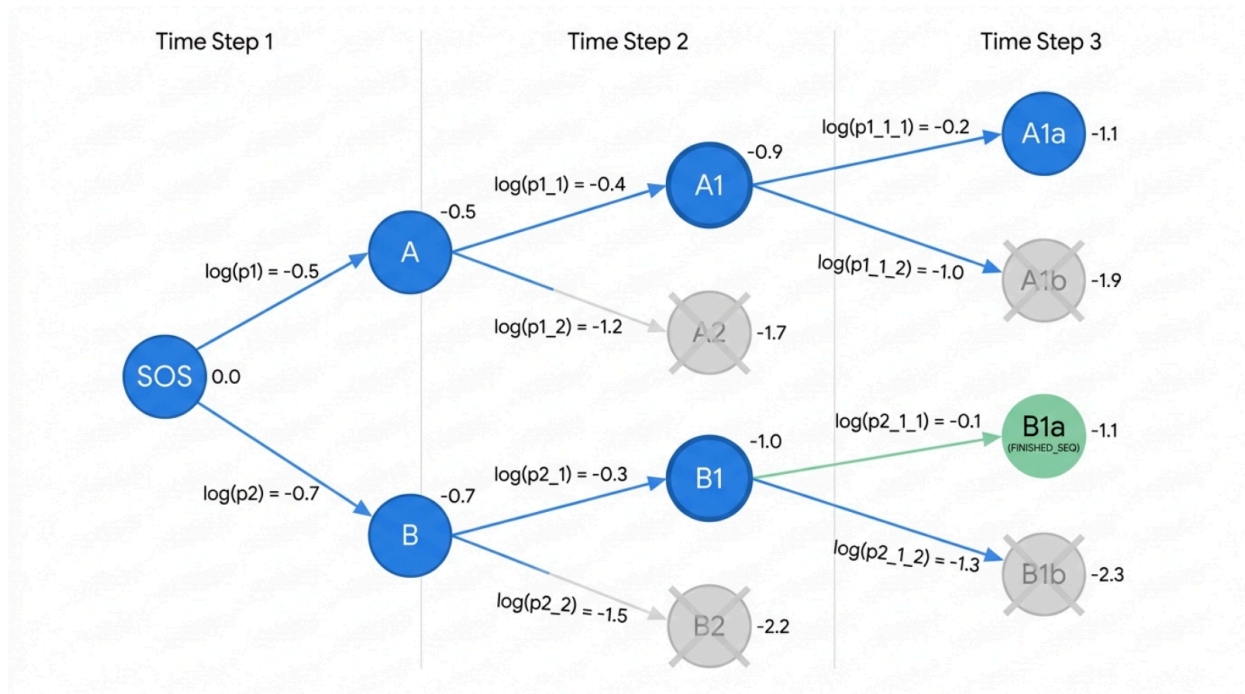
Length Normalization Mechanics

A raw summation of log-probabilities inherently penalizes longer sequences, as every added token introduces a negative log-probability value (since $0 < P \leq 1$, $\log P \leq 0$). To prevent the search algorithm from disproportionately favoring unnaturally short, abrupt translations, length normalization is applied to the final scores.³⁶

The standard penalty formulation applied to sequence y is: $L(y) = \frac{(5+|y|)^\alpha}{(5+1)^\alpha}$ where $|y|$ is the current sequence length and α is a tunable length penalty parameter (commonly set between 0.6 and 0.7).³⁶

During the expansion phase, the algorithm calculates the next-step probabilities, adds them vectorially to ACTIVE_LOG_PROBS, flattens the resulting $k \times |V|$ array, and utilizes np.argpartition or np.argsort to efficiently isolate the top- k indices. Using modulo arithmetic, these flattened indices are projected back to determine exactly which parent beam was extended and which specific vocabulary token was appended.³⁸

Beam Search Hypothesis Pruning



At each decoding step, the algorithm expands all active sequences but prunes the search space down to the top-k highest scoring branches (beam width), terminating when an EOS token is generated.

Name: beam_search

Description: Generates top-k optimal generative sequences using a framework-agnostic, length-normalized beam search iteration loop.

Source: <https://gist.github.com/9ecc17adff1a8793b556f42ffb6a5b19> (NumPy Beam Search adaptation)

License: MIT

Concept type: searching

Signature: (log_probs_fn: Callable, start_token: int, end_token: int, beam_width: int, max_length: int, alpha: float = 0.7) -> list[tuple[list[int], float]]

Pure function boundary: purely numeric integer arrays and probabilities in; sorted list of (token_list, normalized_score) out. The callable log_probs_fn encapsulates all model state externally.

Contracts:

- require: beam_width > 0 and max_length > 0

- require: log_probs_fn returns log-probabilities shape (batch_size, vocab_size)
 - ensure: returned list is sorted by descending length-normalized score
 - ensure: length of returned sequences <= max_length
- Witness: start_token 0, end_token 1, beam 2. If log_probs_fn strictly favors token 5 then 1, returns [(, -0.1),...]
- Dependencies: numpy
- CDG stages covered: bms_molecular/beam_decoding

High-Cardinality Feature Hashing

When modeling text data heavily saturated with massive vocabularies or highly variable categorical permutations—such as in Outbrain's Click Prediction tasks—constructing an exhaustive dictionary mapping strings to integers becomes structurally disastrous.⁴¹

Maintaining a global frequency dictionary in memory causes catastrophic Out-Of-Memory (OOM) failures and breaks the stateless requirement of distributed NLP pipelines.

The algorithmic antidote is Feature Hashing, colloquially known as the "hashing trick." This technique bypasses dictionary storage entirely by feeding text strings directly through a deterministic hash function, mapping them into a fixed-size integer array or sparse matrix using a modulo operation.⁴¹

MurmurHash3 and the Avalanche Effect

The mathematical core of modern feature hashing is the 32-bit signed version of MurmurHash3 (MurmurHash3_x86_32).⁴² Designed by Austin Appleby, MurmurHash is a non-cryptographic hash function engineered specifically for high-speed, general-purpose hash-based lookups.⁴² It heavily utilizes bitwise operations—specifically multiply (MU) and rotate (R) instructions within its inner loop—to achieve extraordinary throughput.

Crucially, MurmurHash3 exhibits a nearly perfect avalanche effect.⁴² An optimal avalanche effect ensures that flipping a single bit in the input string (e.g., changing "apple" to "apply") drastically changes the output hash value, uniformly distributing the hashed integers across the designated feature space.⁴² This uniform distribution is vital to mitigate hash collisions. The

collision probability P in a feature space of size N with k unique features is approximated by the Birthday Paradox formula: $P \approx 1 - e^{-k^2/(2N)}$. To keep collisions negligible, N must be exceptionally large, typically a power of two such as 2^{20} (1,048,576).⁴¹

Cython Optimization vs. Pure Python

Scikit-Learn implements FeatureHasher internally using heavily optimized Cython bindings for MurmurHash3, yielding blazing execution speeds.⁴¹ While pure Python implementations of the algorithm exist (such as pymmh3), the iterative bit-shifting required inside the Python interpreter severely degrades performance, often running hundreds of times slower on large

matrices.⁴⁷ If the CDG infrastructure permits C-extensions, an integration like fmmh3 or mmh3 is highly recommended, as it bypasses the pure Python loop overhead.⁴⁸

Architecturally, the `feature_hash` atom must avoid wrapping Scikit-Learn's full `FeatureHasher` class globally, as Scikit-Learn abstracts away document-level iteration and constructs monolithic sparse matrices that violate memory boundaries in streaming pipelines.⁴¹ Instead, the atomic primitive should define the MurmurHash mapping for a *single, isolated list of tokens*, emitting a sparse dictionary of index-frequency pairs.

Name: `feature_hash`

Description: Maps a sequence of string tokens to an index-frequency sparse array utilizing the `MurmurHash3_x86_32` hashing trick.

Source: <https://github.com/scikit-learn/scikit-learn/blob/master/sklearn/utils/murmurhash.pyx>

License: BSD (via sklearn internals)

Concept type: `data_assembly`

Signature: `(tokens: list[str], n_features: int) -> dict[int, float]`

Pure function boundary: strings in, sparse dictionary of hashed integer indices and feature counts out.

Contracts:

- require: `n_features > 0` (typically heavily padded to a power of 2, e.g., `2**20`)
- ensure: all returned hash keys `k` satisfy `0 <= k < n_features`
- ensure: sum of absolute values in output equals `len(tokens)`
Witness: `["apple", "apple"], n_features 100 -> { 45: 2.0 }` (index derived directly from `murmurhash` modulo)
Dependencies: `mmh3` (C-extension) or `pymmh3` (pure Python fallback)
CDG stages covered: `outbrain/feature_hashing`

N-Gram Feature Extraction

Both word and character n -grams are essential building blocks for capturing local syntactical context, operating under the Markov assumption that the probability of a word depends only on the $n - 1$ preceding elements.⁵⁰ Unigrams treat text as a simple bag-of-words, losing all sequential structure.⁵¹ By expanding features into bigrams and trigrams, the pipeline captures negations ("not good") and compound entities. Furthermore, character n -grams are highly resilient to typographical errors and are exceptionally effective for morphologically rich languages where prefixes and suffixes carry heavy syntactic weight.

Implementing n -gram generation purely in Python negates the need for heavyweight imports

like NLTK or spaCy.⁵¹ The optimal architectural pattern leverages Python's built-in zip function combined with list slicing to generate rolling windows without duplicating memory matrices.⁵³

Word n -grams operate cleanly on lists of pre-tokenized strings, while character n -grams operate directly over the raw text string.⁵³

Name: word_ngrams

Description: Generates contiguous sequential combinations of n items from a tokenized list.

Source: <https://stackoverflow.com/questions/14617601/implementing-ngrams-in-python>

License: CC BY-SA / Public Domain

Concept type: data_assembly

Signature: (tokens: list[str], n: int) -> list[tuple[str,...]]

Pure function boundary: list of strings in, list of sequential string tuples out.

Contracts:

- require: $n > 0$
- ensure: $\text{len}(\text{output}) == \max(0, \text{len}(\text{tokens}) - n + 1)$
- ensure: all output tuples possess an exact length of n
Witness: ["a", "b", "c"], 2 -> [("a", "b"), ("b", "c")]
Dependencies: pure Python
CDG stages covered: multiple/nlp_features

Name: char_ngrams

Description: Extracts overlapping continuous character sequences of length n directly from a string.

Source: Native python algorithmic formulation.

License: Public Domain

Concept type: data_assembly

Signature: (text: str, n: int) -> list[str]

Pure function boundary: single string in, list of fixed-length substring slices out.

Contracts:

- require: $n > 0$
- ensure: $\text{len}(\text{output}) == \max(0, \text{len}(\text{text}) - n + 1)$
- ensure: all output strings possess length n
Witness: "abc", 2 -> ["ab", "bc"]
Dependencies: pure Python

CDG stages covered: multiple/nlp_features

Advanced Text Augmentation and Transduction

Beyond structural extraction and metric calculation, advanced pipelines rely on specialized NLP atoms to drastically alter or augment the text data itself.

Back-Translation Data Augmentation

Deep neural networks in NLP are famously data-hungry and prone to extreme overfitting on restricted datasets.⁵⁵ Data augmentation acts as a mathematical regularizer. While traditional methods like Synonym Replacement (SR) swap isolated words using thesaurus lookups⁵⁵, Back-Translation provides a far more sophisticated syntactic augmentation.⁵⁶

Back-translation conceptually expands a dataset by feeding a source string into a Machine Translation model to convert it to an intermediate target language (e.g., English to French), and then immediately translating that target output back into the source language (French to English).⁵⁷ The resulting round-trip string rigorously preserves the original semantic invariant while introducing entirely novel syntactical structures, grammar distributions, and colloquial variations.⁵⁶

Because pure functions are structurally prohibited from initiating external API calls—which violently breaks CDG determinism and caching—the atom cannot rely on cloud translation services. It must rely on a deterministic translation inference mechanism hosted locally within the execution environment. The function accepts a translation callable as an explicit argument, safely isolating the neural network's state.⁵⁶

Name: back_translate

Description: Augments textual data by generating syntactical variation via a deterministic intermediate-language translation cycle.

Source: <https://github.com/vietai/dab> (Back-translation conceptual framework)

License: MIT

Concept type: data_assembly

Signature: (text: str, translation_fn: Callable[[str, str], str], intermediate_lang: str) -> str

Pure function boundary: input string and deterministic translation callable in, semantically preserved augmented string out.

Contracts:

- require: translation_fn must be a deterministic, immutable mapping
 - ensure: output type is strict string representation
- Witness: "This is great", translation_fn (English->French->English) -> "It is excellent"
- Dependencies: pure Python

CDG stages covered: jigsaw/text_augmentation

Grapheme-to-Phoneme (G2P) Conversion

Transforming written orthography (graphemes) into precise phonetic pronunciation arrays (phonemes) is a mandatory precursor for specific deep learning pipelines, including automated text-to-speech synthesis and complex lyric alignment topologies.⁶¹ Due to historical lexical borrowing and complex spelling rules, English is notoriously inconsistent in its grapheme-to-phoneme mappings.⁶²

A robust, lightweight atom for this task relies primarily on a fast $O(1)$ lookup against the pre-compiled CMU Pronouncing Dictionary.⁶⁴ However, text pipelines inevitably encounter out-of-vocabulary (OOV) tokens and misspellings. The g2p_en library provides an elegant architectural solution. Rather than requiring immense deep learning frameworks like PyTorch or TensorFlow to infer phonemes for OOV words, it utilizes an ultra-lightweight neural network (typically a GRU sequence-to-sequence model) compiled down to pure Numpy or ONNX representations for CPU-bound inference.⁶⁴ This approach emits phonemes corresponding to the standard ARPAbet phonetic set without breaking the dependency constraints of the CDG.⁶⁴

Name: g2p_convert

Description: Transduces English textual orthography into a sequence of ARPAbet phonetic tokens utilizing dictionary lookups and Numpy-bound neural fallbacks.

Source: <https://github.com/kyubyong/g2p>

License: Apache-2.0

Concept type: data_assembly

Signature: (text: str) -> list[str]

Pure function boundary: string in, list of phoneme strings out. Model weights loaded immutably at module scope.

Contracts:

- require: valid string input free of unmappable binary artifacts
- ensure: output strings strictly belong to the standard ARPAbet phonetic set

Witness: "hello" ->

Dependencies: numpy, nltk (strictly for cmudict fallback integration)

CDG stages covered: alice_lyric/g2p

Class-Specific Span Filtering by Length

In the final post-processing stages of Named Entity Recognition or explicit span extraction (such as the Kaggle Feedback Prize task), deep learning models frequently emit spurious,

fragmented, or microscopically short spans that defy linguistic reality.⁶⁶ Applying rigid, class-specific length thresholds serves as a decisive final filter to sanitize the model's output distribution.⁶⁸

By specifying a minimum length dictionary that uniquely maps individual entity classes to integer character or token lengths, the atom efficiently and mathematically purges noisy predictions without relying on complex secondary classification models.⁷⁰

Name: filter_spans_by_length

Description: Systematically removes predicted text spans that mathematically fall below a class-specific minimum character or token length threshold.

Source: Analytical formulation for multi-class thresholding optimization.

License: MIT

Concept type: analysis

Signature: (spans: list[tuple[str, int, int]], min_lengths: dict[str, int]) -> list[tuple[str, int, int]]

Pure function boundary: unverified lists and parameter dicts in, structurally filtered list out.

Contracts:

- require: spans are strictly formatted as (class_name, start_index, end_index)
- require: min_lengths integer values > 0
- ensure: output is a strict subset of input spans
- ensure: for every output span, the length (end_index - start_index) >= min_lengths.get(class_name, 0)

Witness: {"PER": 3} ->

Dependencies: pure Python

CDG stages covered: feedback_prize/span_filtering

Conclusion

The structural integrity of robust machine learning and deep learning NLP pipelines is inextricably bound to the determinism, execution speed, and functional purity of its underlying processing atoms. This exhaustive analysis has mapped the state of the art across text preprocessing, advanced decoding mechanisms, subword token mapping, and complex augmentation logic.

By strictly decoupling pure mathematical and operational logic from framework-heavy state machines—whether by managing Beam Search via highly constrained Numpy invariants or passing immutable translation callables to data augmentation routines—the sciona-atoms repositories guarantee flawless execution repeatability and robust caching. Furthermore, embracing highly compiled MIT-licensed modules such as rapidfuzz for similarity measures directly eliminates the execution bottlenecks inherent in raw Python iteration. By strategically

enforcing these specific functional contracts and integrating these validated algorithms, the CDG architecture achieves unparalleled efficiency, establishing a definitive industry-standard topology for textual processing.

Works cited

1. タグ一覧(ランキング順)【直近1年間/上位25000タグ】【2021/1 更新停止】 - Qiita, accessed April 28, 2026, <https://qiita.com/j5c8k6m8/items/b78a14cb8e1fce4ef6d8>
2. タグ一覧(アルファベット順)【直近1年間/上位25000タグ】【2021/1 更新停止】 - Qiita, accessed April 28, 2026, <https://qiita.com/j5c8k6m8/items/6b15a81204a1d458e392>
3. Qiitaのタグ一覧(アルファベット順), accessed April 28, 2026, <https://qiita.com/j5c8k6m8/items/5601e2f8fbe16887de68>
4. AI Assistant for FAA Regulations - Kaggle, accessed April 28, 2026, <https://www.kaggle.com/code/themuneeb99/ai-assistant-for-faa-regulations>
5. Article 11 - Dataset Curation and Training Language - Cloudurable, accessed April 28, 2026, <https://cloudurable.com/blog/dataset-curation-and-training-language/>
6. Natural Language Processing Practical using Transformers with Python, accessed April 28, 2026, <http://103.203.175.90:81/fdScript/RootOfEBooks/E%20Book%20collection%20-%202024%20-%20H/CSE%20%20IT%20AIDS%20ML/Natural%20Language%20Processing%20Practical.pdf>
7. pypellchecker - PyPI, accessed April 28, 2026, <https://pypi.org/project/pypellchecker/>
8. How to Write a Spelling Corrector - Peter Norvig, accessed April 28, 2026, <https://norvig.com/spell-correct.html>
9. spell.py - Peter Norvig, accessed April 28, 2026, <https://norvig.com/spell.py>
10. A Simple Guide to the Levenshtein Distance Algorithm (With Python Example) - Medium, accessed April 28, 2026, <https://medium.com/@selinamangaroo/a-simple-guide-to-the-levenshtein-distance-algorithm-with-python-example-e4efeba1383e>
11. Levenshtein Distance: A Comprehensive Guide - DigitalOcean, accessed April 28, 2026, <https://www.digitalocean.com/community/tutorials/levenshtein-distance-python>
12. A simple Python implementation of the Levenshtein distance algorithm (fuzzy string matching). - GitHub, accessed April 28, 2026, <https://github.com/jamfromouterspace/levenshtein>
13. python - Better fuzzy matching performance? - Stack Overflow, accessed April 28, 2026, <https://stackoverflow.com/questions/21408760/better-fuzzy-matching-performance>
14. rapidfuzz/RapidFuzz: Rapid fuzzy string matching in Python using various string metrics - GitHub, accessed April 28, 2026, <https://github.com/rapidfuzz/RapidFuzz>
15. A Comparative Analysis of Python Text Matching Libraries: A Multilingual Evaluation of Capabilities, Performance and Resource Utilization | International

- Journal of Environment, Engineering and Education, accessed April 28, 2026,
<https://ijeedu.com/index.php/ijeedu/article/view/188>
16. fuzzmatch - PyPI, accessed April 28, 2026, <https://pypi.org/project/fuzzmatch/>
 17. pyjarowinkler · PyPI, accessed April 28, 2026,
<https://pypi.org/project/pyjarowinkler/>
 18. KhanPython/Jaro-Similarity: LuaU Jaro Similarity and Jaro Winkler Implementation - GitHub, accessed April 28, 2026, <https://github.com/KhanPython/Jaro-Similarity>
 19. Text similarity - A Lazy Data Science Guide - Mohit Mayank, accessed April 28, 2026,
http://mohitmayank.com/a_lazy_data_science_guide/natural_language_processing/text_similarity/
 20. Guide to Fuzzy Matching with Python | IBKR Quant, accessed April 28, 2026,
<https://www.interactivebrokers.com/campus/ibkr-quant-news/guide-to-fuzzy-matching-with-python/>
 21. Machine Learning is All You Need: A Simple Token-based Approach for Effective Code Clone Detection - Yueming Wu, accessed April 28, 2026,
https://wu-yueming.github.io/Files/ICSE2024_Toma.pdf
 22. GitHub - tonytonyjan/jaro_winkler: Ruby & C implementation of Jaro-Winkler distance algorithm which supports UTF-8 string., accessed April 28, 2026,
https://github.com/tonytonyjan/jaro_winkler
 23. Give Me 15 Minutes and I will Show You How to Clean Your Messiest Dataset - Medium, accessed April 28, 2026,
<https://medium.com/@BIWave/give-me-15-minutes-and-i-will-show-you-how-to-clean-your-messiest-dataset-2212b15395ad>
 24. GitHub - chakki-works/seqeval: A Python framework for sequence labeling evaluation(named-entity recognition, pos tagging, etc...), accessed April 28, 2026,
<https://github.com/chakki-works/seqeval>
 25. michael_wzhu91/seqeval - Gitee, accessed April 28, 2026,
<https://gitee.com/michael-wzhu91/seqeval>
 26. metrics/seqeval · 00742e0558fe927c20b7285855142cec9ff402ef · apulix / Evaluate - GitLab, accessed April 28, 2026,
<https://gitlab.apulis.com.cn/apulix/evaluate/-/tree/00742e0558fe927c20b7285855142cec9ff402ef/metrics/seqeval>
 27. seqeval/seqeval/metrics/sequence_labeling.py at master · chakki ..., accessed April 28, 2026,
https://github.com/chakki-works/seqeval/blob/master/seqeval/metrics/sequence_labeling.py
 28. 4th place solution in the Feedback Prize Competition 🏆 - GitHub, accessed April 28, 2026, <https://github.com/affiljoo3581/Feedback-Prize-Competition>
 29. Accelerating Topic modeling with RAPIDS and BERT models | by Vibhu Jawa - Medium, accessed April 28, 2026,
<https://medium.com/rapids-ai/accelerating-topic-modeling-with-rapids-and-bert-models-be9909eed2>
 30. Models - Hugging Face, accessed April 28, 2026,
https://huggingface.co/docs/transformers/v4.15.0/main_classes/model

31. Models - Hugging Face, accessed April 28, 2026,
https://huggingface.co/docs/transformers/v4.22.2/main_classes/model
32. How to Fine-tune an LLM Part 3: The HuggingFace Trainer | alpaca_ft - Wandb, accessed April 28, 2026,
https://wandb.ai/capecape/alpaca_ft/reports/How-to-Fine-tune-an-LLM-Part-3-The-HuggingFace-Trainer--Vmlldzo1OTEyNjMy
33. Martin's blog - Fine-tuning a Transformer for text classification, accessed April 28, 2026, https://7c0h.com/blog/new/fine_tuned_text_classif.html
34. Feedback Prize - English Language Learning | Kaggle, accessed April 28, 2026,
<https://www.kaggle.com/competitions/feedback-prize-english-language-learning/discussion/351577>
35. This is the solution for 2nd rank in Kaggle competition: Feedback Prize - Evaluating Student Writing. - GitHub, accessed April 28, 2026,
<https://github.com/ubamba98/feedback-prize>
36. Easy to understand implementation of beam search algorithm used ..., accessed April 28, 2026, <https://gist.github.com/9ecc17adff1a8793b556f42ffb6a5b19>
37. Machine Translation Decoding beyond Beam Search - ACL Anthology, accessed April 28, 2026, <https://aclanthology.org/2021.emnlp-main.662.pdf>
38. How to Implement a Beam Search Decoder for Natural Language Processing - MachineLearningMastery.com, accessed April 28, 2026,
<https://machinelearningmastery.com/beam-search-decoder-natural-language-processing/>
39. Beam Search Decoding Demystified: Code Review and Real-World Applications, accessed April 28, 2026,
<https://eliashossain9111.medium.com/beam-search-decoding-demystified-code-review-and-real-world-applications-8060ca3a50b6>
40. Stealing the Decoding Algorithms of Language Models - arXiv, accessed April 28, 2026, <https://arxiv.org/pdf/2303.04729>
41. FeatureHasher — scikit-learn 1.8.0 documentation, accessed April 28, 2026,
https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.FeatureHasher.html
42. Scikit-learn's FeatureHasher - Medium, accessed April 28, 2026,
<https://medium.com/@cwakhusama/test-d5bee5d2e840>
43. murmurhash3_32 — scikit-learn 1.8.0 documentation, accessed April 28, 2026,
https://scikit-learn.org/stable/modules/generated/sklearn.utils.murmurhash3_32.html
44. mmh3 documentation - Read the Docs, accessed April 28, 2026,
https://mmh3.readthedocs.io/_/downloads/en/latest/epub/
45. Murmur3 hash in C - GitHub, accessed April 28, 2026,
<https://github.com/PeterScott/murmur3>
46. Javascript implementation of Murmurhash3 to give the same result as Murmurhash3.cpp used by transform available in Python's sklearn - Stack Overflow, accessed April 28, 2026,
<https://stackoverflow.com/questions/39836589/javascript-implementation-of-murmurhash3-to-give-the-same-result-as-murmurhash3>

47. wc-duck/pymmh3: a pure python MurmurHash3 implementation. - GitHub, accessed April 28, 2026, <https://github.com/wc-duck/pymmh3>
48. dream2333/PyFastMurmurHash3: A Python extension module of MurmurHash3 developed using a mix of C language and Cython. - GitHub, accessed April 28, 2026, <https://github.com/dream2333/PyFastMurmurHash3>
49. mmh3 - PyPI, accessed April 28, 2026, <https://pypi.org/project/mmh3/>
50. Generating Word N-grams Using Basic Python in Linguistics | by Lingadeu - Medium, accessed April 28, 2026, <https://medium.com/@lingostat/generating-word-n-grams-using-basic-python-in-linguistics-c533c1ea9bd8>
51. What Are N-Grams and How to Implement Them in Python? - Analytics Vidhya, accessed April 28, 2026, <https://www.analyticsvidhya.com/blog/2021/09/what-are-n-grams-and-how-to-implement-them-in-python/>
52. beyzanc/n-gram-language-modeling - GitHub, accessed April 28, 2026, <https://github.com/beyzanc/n-gram-language-modeling>
53. Implementing ngrams in Python - Stack Overflow, accessed April 28, 2026, <https://stackoverflow.com/questions/14617601/implementing-ngrams-in-python>
54. Implementation of n-grams in python code for multi-class text classification - Stack Overflow, accessed April 28, 2026, <https://stackoverflow.com/questions/55555159/implementation-of-n-grams-in-python-code-for-multi-class-text-classification>
55. Popular Data Augmentation Techniques in NLP - Paperspace Blog, accessed April 28, 2026, <https://blog.paperspace.com/data-augmentation-for-nlp/>
56. Unraveling Sarcasm in Arabic with AraBERT: A Comprehensive Guide from Data Preprocessing to Prediction | by Rehab Reda | Medium, accessed April 28, 2026, <https://medium.com/@rehabreda/unraveling-sarcasm-in-arabic-with-arabert-a-comprehensive-guide-from-data-preprocessing-to-a4dc7e30b39d>
57. Text data augmentation with Back Translation - Deep Learning, accessed April 28, 2026, <https://dzlab.github.io/dltips/en/pytorch/text-augmentation/>
58. vietai/dab: Data Augmentation by Backtranslation (DAB) \(\cdot_{-}\) - GitHub, accessed April 28, 2026, <https://github.com/vietai/dab>
59. A Comprehensive Study on NLP Data Augmentation for Hate Speech Detection: Legacy Methods, BERT, and LLMs - arXiv, accessed April 28, 2026, <https://arxiv.org/pdf/2404.00303?>
60. Notes on Transformers Book Ch. 9 - Christian Mills, accessed April 28, 2026, <https://christianjmills.com/Notes-on-Transformers-Book-09/>
61. Grapheme-to-Phoneme Models — NVIDIA NeMo Framework User Guide, accessed April 28, 2026, <https://docs.nvidia.com/nemo-framework/user-guide/latest/nemotoolkit/tts/g2p.html>
62. A Survey of Grapheme-to-Phoneme Conversion Methods - MDPI, accessed April 28, 2026, <https://www.mdpi.com/2076-3417/14/24/11790>
63. Grapheme-to-Phoneme Conversion (G2P) - Deepgram, accessed April 28, 2026, <https://deepgram.com/ai-glossary/grapheme-to-phoneme-conversion-g2p>

64. GitHub - tronghieuit/tiny-tts: The Smallest English TTS Model with only 1M parameters, accessed April 28, 2026, <https://github.com/tronghieuit/tiny-tts>
65. GitHub - Kyubyong/g2p: g2p: English Grapheme To Phoneme Conversion, accessed April 28, 2026, <https://github.com/kyubyong/g2p>
66. How to use classification threshold to balance precision and recall - Evidently AI, accessed April 28, 2026, <https://www.evidentlyai.com/classification-metrics/classification-threshold>
67. A Gentle Introduction to Threshold-Moving for Imbalanced Classification - MachineLearningMastery.com, accessed April 28, 2026, <https://machinelearningmastery.com/threshold-moving-for-imbalanced-classification/>
68. How to set separate threshold for each class(multiclass) in model.predict array in keras....any help will be appreciated - Stack Overflow, accessed April 28, 2026, <https://stackoverflow.com/questions/73311877/how-to-set-separate-threshold-for-each-classmulticlass-in-model-predict-array>
69. Tip for Classification with Python: Leverage Class Probability Thresholds for Enhanced Model Performance | by Waseem Kathia | Medium, accessed April 28, 2026, <https://medium.com/@waseem.kathia/tip-for-classification-with-python-leverage-class-probability-thresholds-for-enhanced-model-2e2a753c8b69>
70. How to adjust the classification thresholds in a multiclass classification problem?, accessed April 28, 2026, <https://stats.stackexchange.com/questions/611695/how-to-adjust-the-classification-thresholds-in-a-multiclass-classification-probl>